Revision — 1.4.3 Boolean Algebra

OCR H446 — one-page revision summary

Must know

- **Gates:** NOT ¬A (invert); AND `A·B` (both 1); OR `A+B` (either 1); XOR `A⊕B` (inputs **differ**); NAND `¬(A·B)`; NOR `¬(A+B)`. **NAND and NOR are universal** — any circuit can be built from either alone.
- **De Morgan's laws:** `¬(A·B) = ¬A + ¬B` and `¬(A+B) = ¬A · ¬B`. "Break the bar, change the sign" — negate **each** variable **and** swap AND ↔ OR.
- **Core identities:** Identity `A·1=A`, `A+0=A`; Null `A·0=0`, `A+1=1`; Idempotent `A·A=A`, `A+A=A`; Complement `A·Ā=0`, `A+Ā=1`; Double negation `¬(¬A)=A`.
- **Distributive:** `A·(B+C) = A·B + A·C` and `A+(B·C) = (A+B)·(A+C)`. **Absorption:** `A + A·B = A` and `A·(A+B) = A`.
- **Simplify** expressions by applying identities step by step; **tidy** with double negation. Use to reduce gate count.
- **Karnaugh (K-)maps:** plot outputs, group adjacent **1s** in blocks that are **powers of two** (1, 2, 4, 8…), as **large** as possible, may wrap/overlap → minimal sum-of-products.
- **Half adder** (2 bits, no carry-in): **Sum = A ⊕ B**, **Carry = A · B**.
- **Full adder** (A, B, Cin) = two half adders + OR: **Sum = A ⊕ B ⊕ Cin**, **Cout = (A·B) + (Cin·(A⊕B))**. Chain them → **ripple-carry adder** for multi-bit addition.
- **D-type flip-flop:** edge-triggered **1-bit memory** (sequential logic). On the active **clock edge** it stores input **D** and presents it at **Q** until the next edge; synchronises data to the clock. Uses: registers, counters, memory cells.
- **Combinational** logic = output depends only on current inputs; **sequential** logic = output depends on inputs **plus stored state**.

Key terms

Term	Definition
XOR	Gate outputting 1 only when its two inputs differ
Universal gate	Gate (NAND or NOR) from which any circuit can be built
De Morgan's law	`¬(A·B)=¬A+¬B`, `¬(A+B)=¬A·¬B` — negate each term and swap operator
Absorption law	`A + A·B = A`; `A·(A+B) = A`
Karnaugh map	Grid grouping adjacent 1s (powers of 2) to minimise an expression
Half adder	Adds 2 bits → Sum (A⊕B), Carry (A·B); no carry-in
Full adder	Adds 3 bits (A, B, Cin) → Sum + Cout; chainable
D-type flip-flop	Edge-triggered 1-bit memory storing D on a clock edge

Worked example / how to

Simplify `¬(¬A + B)` using De Morgan's law + double negation: - Apply De Morgan's to the outer bar (break bar, change + to ·, negate each term): `¬(¬A + B) = ¬(¬A) · ¬B`. - Apply double negation `¬(¬A)`

`= A`: → `A · ¬B`. - **Result: `A · ¬B`.**

Exam tips

- De Morgan's: negate **each** variable **and** change the operator — not just one side; finish by tidying double negations.
- State **Sum = A⊕B** and **Carry = A·B** exactly; the half adder has **no carry-in**, the full adder adds **three** bits.
- For simplification, **name the law** used at each step (e.g. absorption, distributive) — it earns marks.
- K-map groups must be **powers of two** and **as large as possible**; they may wrap around and overlap.